

CUSUM RAP TOOL

Developer and Administrator Guide

Architecture, configuration, deployment, and maintenance

Documentation revision	June 2026
Language	English
Status	Aligned with the supplied current application source

Purpose. This document describes the current workflow and implementation represented by the supplied project files. It replaces older instructions that referred to previous navigation, storage, or output structures.

Contents

1. Current project structure

- Repository map
- Module responsibilities

2. Local execution and dependencies

- Run modes
- Required packages

3. Configuration

- Top-level settings
- Country hierarchies
- CUSUM validation

4. Import and preparation pipeline

- Reading and aliases
- Overlap processing
- Weekly completion and coverage

5. CUSUM implementation

- Expected counts
- k and h
- Alarm calculation

6. UI, tables, and downloads

- Display settings
- Filtering
- Output contracts

7. Testing and validation

- Automated tests
- Manual checks

8. Deployment and operations

- Web
- Desktop/offline
- Security

9. Maintenance checklist

- Before release
- Troubleshooting

1. Current project structure

The application is a modular R Shiny project. `global.R` loads configuration and recursively sources the operational R files, excluding `R/scripts`. `app.R` mounts the main CUSUM module with the id `main_analysis`.

```
CUSUM-RAP/
|-- app.R
|-- global.R
|-- config.json
|-- CUSUM-RAP.Rproj
|-- R/
|   |-- help_ui.R
|   |-- shared/
|   |   |-- api_pop_io.R
|   |   |-- config_helpers.R
|   |-- cusum/
|   |   |-- functions_cusum.R
|   |   |-- functions_io.R
|   |   |-- functions_parameters.R
|   |   |-- functions_plot.R
|   |   |-- mod_cusum.R
|   |   |-- mod_cusum/
|   |       |-- uploads.R      |-- selectors.R
|   |       |-- inputs.R      |-- plots.R
|   |       |-- tables.R      |-- summary.R
|   |       |-- downloads.R   |-- helpers.R
|   |-- scripts/
|-- tests/
|-- www/
|   |-- config.css
|   |-- docs/
```

Component	Responsibility
<code>mod_cusum.R</code>	Top-level UI and module coordinator.
<code>uploads.R</code>	Uploaded-file state, country selection, overlap choice, and raw combined data.
<code>selectors.R</code>	Location filters and hierarchical Detailed View navigation.
<code>inputs.R</code> / <code>helpers.R</code>	Parameter UI, calculated k, recommended h, table-filter helpers, and display settings.
<code>summary.R</code>	Prepared dataset, overlap processing, coverage summaries, and change log.
<code>plots.R</code>	CUSUM execution and plot rendering.
<code>tables.R</code>	Reference, weekly, and prepared-data tables.
<code>downloads.R</code>	CSV and plot downloads.
<code>shared/api_pop_io.R</code>	Common file reading, aliases, standardisation, logs, and validation.
<code>shared/config_helpers.R</code>	Country selection, hierarchy choices, and configuration validation.

2. Local execution and dependencies

1. Open `CUSUM-RAP.Rproj` in RStudio.
2. Restore or install the required packages.
3. Open `app.R` and select Run App, or execute `shiny::runApp()` from the project root.

```
shiny::runApp()
```

Area	Packages used by the supplied source
Shiny/UI	shiny, shinyjs, DT, jsonlite
Data	dplyr, tidyr, purrr, rlang, readr, stringr
Dates	lubridate, ISOweek
Plots	ggplot2
Utilities	here

Working directory. Run from the project root so config.json, R/, and www/ resolve through the relative paths used by global.R.

3. Configuration

config.json is loaded before the application modules. The current supplied configuration contains is_offline, active_country, a cusum block, and one hierarchy block per country.

Key	Purpose
is_offline	Application-level offline flag retained for shared deployment conventions.
active_country	Default country. The helper can also expose a selector when several active countries are configured.
Country.levels	Ordered labels corresponding internally to country, level1, level2, level3, and level4.
cusum.default_detection_period	Default Detection Period shown in the sidebar.
cusum.minimum_prepared_weeks	Minimum prepared time span accepted by coverage validation.
cusum.minimum_observed_coverage_stop	Lowest observed-week coverage allowed before analysis stops.
cusum.minimum_observed_coverage_warn	Coverage below this value produces a warning.

```
{
  "active_country": "Argentina",
  "cusum": {
    "default_detection_period": 52,
    "minimum_prepared_weeks": 52,
    "minimum_observed_coverage_stop": 0.4,
    "minimum_observed_coverage_warn": 0.7,
    "fill_missing_weeks_with_zero": true
  },
  "Argentina": {
    "levels": ["country", "province", "department", "fraction"]
  }
}
```

Hierarchy contract. The order of levels determines which generic input column is used. The first item maps to country, the second to level1, the third to level2, and so on.

4. Import and preparation pipeline

4. api_pop_read_file() detects comma or semicolon delimiters.
5. api_pop_standardise_columns() normalises aliases, types, and geographic values.

6. Active files are combined with a file index and validated.
7. `process_target_data()` filters the selected units, resolves overlaps, aggregates weekly counts, and completes time series.
8. Preparation metadata and a human-readable log are attached to the returned data.

CUSUM requires year, week, a case-count column, and the generic column for the chosen geography. Population is required for manual μ_0 and optional for automatic μ_0 .

Overlap method	Implementation intent
new	Keep the newest file index for duplicate unit/week records.
old	Keep the oldest file index.
sum	Aggregate duplicates by summing case counts.

Coverage is assessed per analysis unit. The minimum accepted time span is $\max(\text{minimum_prepared_weeks}, \text{selected detection period})$. Observed coverage below the stop threshold blocks analysis; coverage below the warning threshold allows analysis with a warning.

Zero filling. `fill_missing_weeks_with_zero` is intended for true zero-case weeks. Do not use it to conceal reporting gaps. The preparation log and coverage table should remain visible to analysts.

5. CUSUM implementation

`cusum_core()` implements a one-sided upper CUSUM. With the standard transformation, $z_t = (y_t - \mu_t) / \sqrt{\mu_t}$. The running statistic is $S_t = \max(0, S_{t-1} + z_t - k)$. An alarm is raised when $S_t \geq h$.

`run_cusum_for_unit()` supports either a fixed expected count or a Poisson GLM prediction. Parameters may be scalar or named vectors resolved by `analysis_unit_id`. `run_cusum_all_units()` applies the calculation across units.

Parameter	Implementation
μ / μ_{hat}	Expected weekly counts. Automatic values are estimated per unit; manual rates per 100,000 are converted using population.
RR	Passed to <code>calculate_k_from_rr()</code> together with the expected count.
k	May be a scalar or a named unit-level vector.
h	User-entered threshold; <code>recommend_h()</code> displays a suggested value from ARL0 and k.
reset	The current UI execution uses <code>reset = TRUE</code> so accumulation restarts after an alarm.

Terminology. The engine uses expected counts. Manual μ_0 is entered as a weekly rate per 100,000 and converted to manual $\mu_0 r$ a unit-specific count using population.

6. UI, tables, and downloads

The Help tab owns the font-size and decimal-place inputs. Font selection also changes plot height. Decimal formatting is applied to on-screen DT tables only.

- Overview: alarm heatmap.
- Detailed View: hierarchical unit navigation, selection summary, observed/expected plot, and CUSUM process plot.
- Analysis Results: unit-level reference table plus weekly result table.
- Prepared Data: coverage status, change log, summary, table, and download.

Download handlers should use the unformatted analysis objects so CSV files retain full precision. Plot downloads should use the same selected unit and display settings as the visible detailed plots.

7. Testing and validation

The repository includes tests/testthat.R, tests/TESTING.md, and testthat files. Run tests from the project root after changes to import aliases, preparation, configuration, parameters, or plotting.

```
testthat::test_dir("tests/testthat")
```

- Upload one and multiple CSV files.
- Exercise all overlap choices.
- Test an empty location selection and a restricted selection.
- Verify stop and warning coverage thresholds.
- Compare automatic and manual μ_0 .
- Confirm downloads enable only after successful analysis.
- Check table filters and 0-8 displayed decimal places.

8. Deployment and operations

The app can be deployed with shinyapps.io, Posit Connect, Shiny Server, or a controlled desktop wrapper. Deployment packaging must include config.json, R/, www/, and all required R packages.

- Use an approved hosting environment for restricted epidemiological data.
- Set request and upload limits appropriate to the expected file sizes.
- Retain server logs without writing uploaded row-level data unless explicitly approved.
- Validate the project root and case-sensitive documentation filenames on Linux.

Documentation links. The running standalone app opens docs/User_Manual.pdf. Keep that legacy filename present even when additional language-specific manuals are added.

9. Maintenance checklist

- Run automated tests.
- Start the app from a clean R session.
- Verify config validation messages.
- Test each tab and download.
- Confirm user and developer manuals match the current UI.
- Check PDFs on a case-sensitive deployment and confirm no stale Documentation.pdf references remain.

Symptom	Likely area
App fails during startup	Package installation, config.json parsing, or recursive source order.
Country or level choices are wrong	config.json levels and rap_cusum_geo_choices().
Import aliases fail	R/shared/api_pop_io.R.
Prepared weeks or overlaps are wrong	process_target_data() and summary.R.
k/h recommendations fail	functions_parameters.R and inputs.R.
Charts or downloads differ	functions_plot.R, plots.R, and downloads.R.